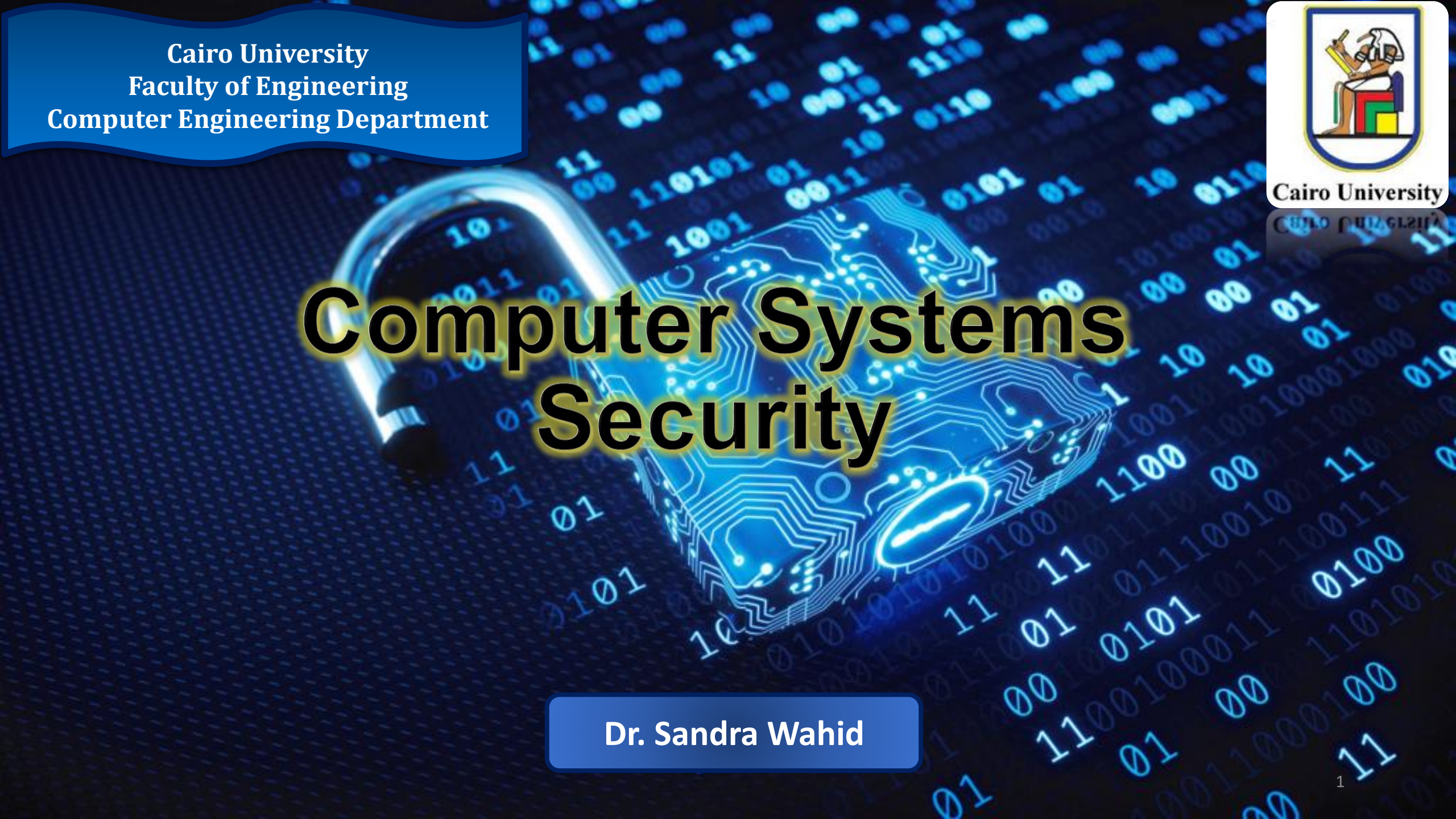
Cairo University
Faculty of Engineering
Computer Engineering Department

Cairo University

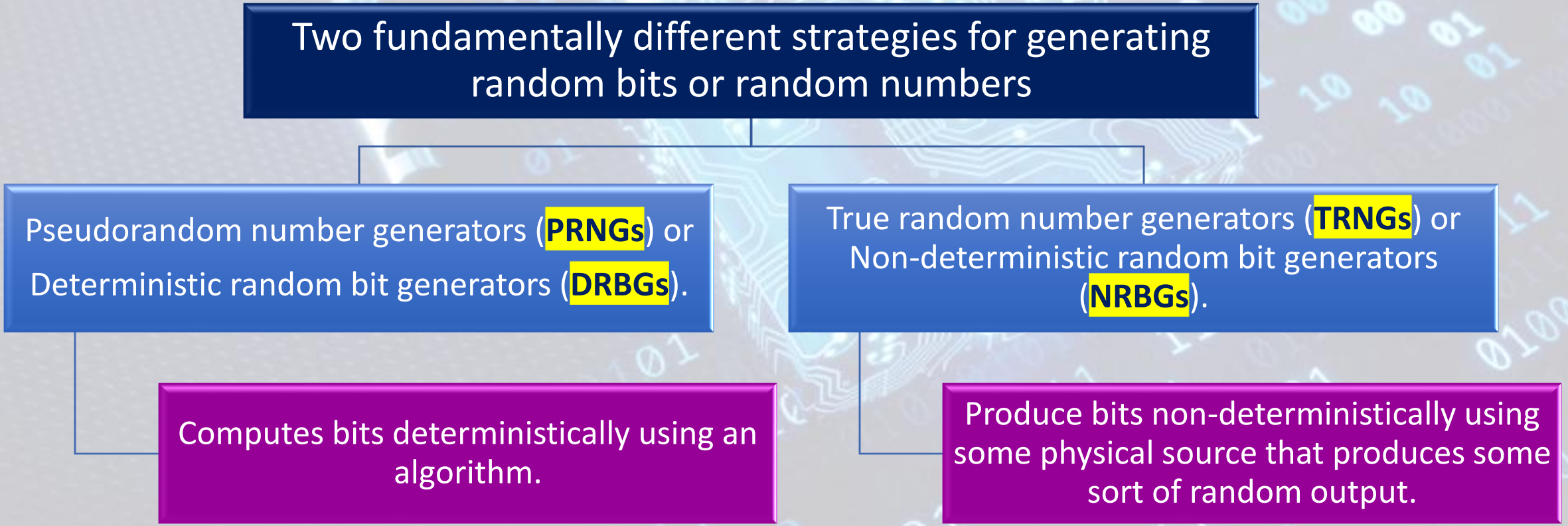


Computer Systems Security

Dr. Sandra Wahid

Random Bits Streams

- An **important cryptographic function** is the generation of random bit streams.
- Random bits streams are used in a wide variety of contexts, including **key generation** and **encryption**.



Random Numbers

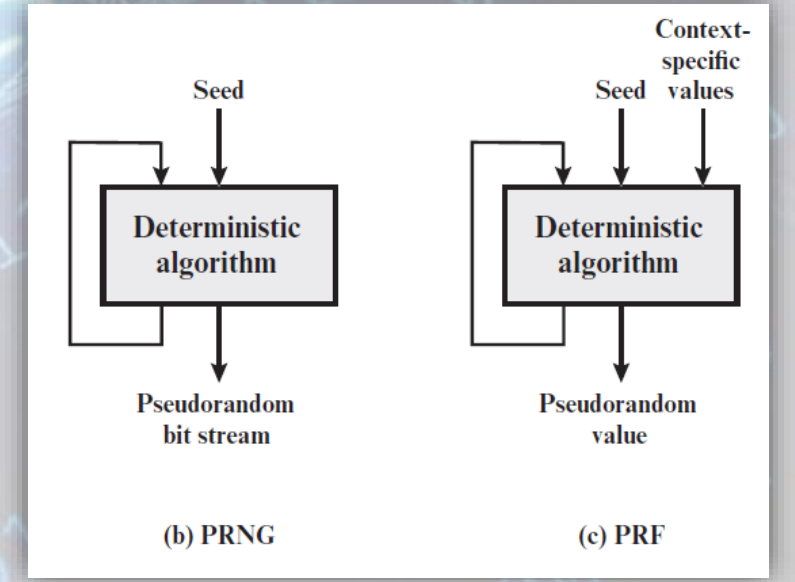
- Many uses of random numbers in cryptography
  - Nonces.
  - Session keys.
  - Public keys.
  - Keystreams for symmetric stream encryption.

- Requirements for random numbers: **randomness** and **unpredictability**

1. **Randomness:** two criteria are used to validate randomness:
   - Uniform distribution: the frequency of occurrence of ones and zeros should be approximately equal.
   - Independence: No one subsequence in the sequence can be inferred from the others.
2. **Unpredictability:** with "true" random sequences, each number is statistically independent of other numbers in the sequence and therefore unpredictable.

Pseudorandom Number Generators (PRNGs)

- Pseudorandom numbers: appear to be statistically random, despite having been produced by a completely deterministic and repeatable process.
- PRNG takes as input a fixed value, called the **seed**, and produces a sequence of output bits using a **deterministic** algorithm. Typically, there is some feedback path by which some of the results of the algorithm are fed back as input as additional output bits are produced.
- Note that the output bit stream is determined solely by the input value(s).
  - So an adversary who knows the algorithm and the seed can reproduce the entire bit stream.
- Two different forms of PRNGs, based on application:
  - Pseudorandom number generator (PRNG)
  - Pseudorandom function (PRF): the PRF takes as input a seed plus some **context specific values**, such as a user ID or an application ID



PRNG Requirements

- The basic requirement is that an adversary who **does not know the seed** is unable to determine the pseudorandom string.
- Therefore, specific requirements in the areas of randomness, unpredictability, and the characteristics of the seed exist:

Randomness: the tests should establish the following three characteristics:

1. **Uniformity:** The occurrence of a zero or one is equally likely, that is, the probability of each is exactly 1/2. The expected number of zeros (or ones) is n/2, where n = the sequence length.
2. **Scalability:** If a sequence is random, then any extracted subsequence should also be random.
3. **Consistency:** The behavior of a generator must be consistent across different starting values (seeds).

PRNG Requirements

Unpredictability: a stream of pseudorandom numbers should exhibit two forms of unpredictability:

1. **Forward unpredictability:** If the seed is unknown, the next output bit in the sequence should be unpredictable in spite of any knowledge of previous bits in the sequence.
2. **Backward unpredictability:** It should also not be feasible to determine the seed from knowledge of any generated values. No correlation between a seed and any value generated from that seed should be evident.

Seed Requirements: For cryptographic applications, the seed that serves as input to the PRNG must be secure → In fact, the seed itself must be a random or pseudorandom number.

Cryptographic PRNGs Algorithm Design

- Two categories for cryptographic PRNGs algorithm design:

1. **Purpose-built algorithms:** These are algorithms designed specifically and solely for the purpose of generating pseudorandom bit streams.
2. **Algorithms based on existing cryptographic algorithms:** Cryptographic algorithms have the effect of randomizing input data (indeed, this is a requirement of such algorithms) → Cryptographic algorithms can be used to create PRNGs.

Purpose-built algorithms:	• Linear Congruential Generators • Blum Blum Shub Generator
Algorithms based on existing cryptographic algorithms:	• PRNG Using Block Cipher Modes of Operation • ANSI X9.17 PRNG

Linear Congruential Generators

- The algorithm is parameterized with four numbers:

m	the modulus	$m > 0$
a	the multiplier	$0 < a < m$
c	the increment	$0 \leq c < m$
X_0	the starting value, or seed	$0 \leq X_0 < m$

- The sequence of random numbers {Xn} is obtained via the following **iterative** equation:

$$X_{n+1} = (aX_n + c) \bmod m$$

- Examples:
  - values a = 7, c = 0, m = 32, and X_0 = 1 → sequence {7, 17, 23, 1, 7, etc.}
    - Of the 32 possible values, only 4 are used→ unsatisfactory →the sequence has a period of 4.
  - values a = 5, c = 0, m = 32, and X_0 = 1 → sequence {5, 25, 29, 17, 21, 9, 13, 1, 5, etc. }
    - →the sequence has a period of 8.

Linear Congruential Generators

- We would like **m to be very large**, so that there is the potential for producing a long series of distinct random numbers.
  - A common criterion is that m be nearly equal to the maximum representable non-negative integer for a given computer.
  - Typically, a value of m near to or equal to 2^{31} is chosen.
- The function should be a **full-period generating function** → generate all the numbers from **0 through m - 1 before repeating**.
- If *m* is prime and *c = 0*, then for certain values of *a* the period of the generating function is *m-1*, with only the value *0* missing →For 32-bit arithmetic, a convenient prime value of *m* is $2^{31} - 1$.
- The generating function becomes: $X_{n+1} = (aX_n) \bmod (2^{31} - 1)$
- A widely used value for *a* is $7^5 = 16807$

Linear Congruential Generators

- If an opponent knows that the linear congruential algorithm is being used and if the parameters are known (e.g., a = 7^5, c = 0, m = $2^{31} - 1$), then once a single number is discovered, all subsequent numbers are known.
- Even if the opponent knows only that a linear congruential algorithm is being used, knowledge of a small part of the sequence is sufficient to determine the parameters of the algorithm:
  - Suppose that the opponent is able to determine values for *X0*, *X1*, *X2*, and *X3*.
  - These equations can be solved for *a*, *c*, and *m*.

$$X_1 = (aX_0 + c) \bmod m$$
$$X_2 = (aX_1 + c) \bmod m$$
$$X_3 = (aX_2 + c) \bmod m$$

- <u>Possible Solutions:</u> using an internal system clock to modify the random number stream.
  - One way is to restart the sequence after every *N* numbers using the ***current clock value (mod m)*** as the new seed.
  - Another way would be simply to add the ***current clock value to each random number (mod m).***

Blum Blum Shub Generator (BBS)

- The BBS is referred to as a cryptographically secure pseudorandom bit generator (**CSPRBG**).
- First, choose two large prime numbers, *p* and *q*, that both have a remainder of 3 when divided by 4: $p \equiv q \equiv 3 \pmod 4$ → *p mod 4 = q mod 4 = 3*
  - Example of small numbers: *p=7* and *q=11*
- Let n = p * q, next, choose a random number *s*, such that *s* is relatively prime to *n* → neither *p* nor *q* is a factor of *s*.
  - Two integers *a* and *b* are relatively prime if their only common positive integer factor is 1
  - Greatest Common Divisor *GCD(a,b)=1*.
- Then the BBS generator produces a sequence of bits B_i according to the following algorithm:

$$X_0 = s^2 \bmod n$$
$$\textbf{for } i = 1 \textbf{ to } \infty$$
$$X_i = (X_{i-1})^2 \bmod n$$
$$B_i = X_i \bmod 2$$

→ the least significant bit is taken at each iteration.

Blum Blum Shub Generator (BBS)

- The security of BBS is based on the difficulty of factoring *n*

  →given *n*, we need to determine its two prime factors *p* and *q*.

- Example: n = 192649 = 383 * 503, and the seed s = 101355.

$$X_0 = s^2 \bmod n$$
$$\textbf{for } i = 1 \textbf{ to } \infty$$
$$X_i = (X_{i-1})^2 \bmod n$$
$$B_i = X_i \bmod 2$$

i	X_i	B_i
0	20749	
1	143135	1
2	177671	1
3	97048	0
4	89992	0
5	174051	1
6	80649	1
7	45663	1
8	69442	0
9	186894	0
10	177046	0

i	X_i	B_i
11	137922	0
12	123175	1
13	8630	0
14	114386	0
15	14863	1
16	133015	1
17	106065	1
18	45870	0
19	137171	1
20	48060	0

PRNG Using Block Cipher Modes of Operation

- A popular approach to PRNG construction is to use a symmetric block cipher as the heart of the PRNG mechanism. For any block of plaintext, a symmetric block cipher produces an output block that is apparently random.

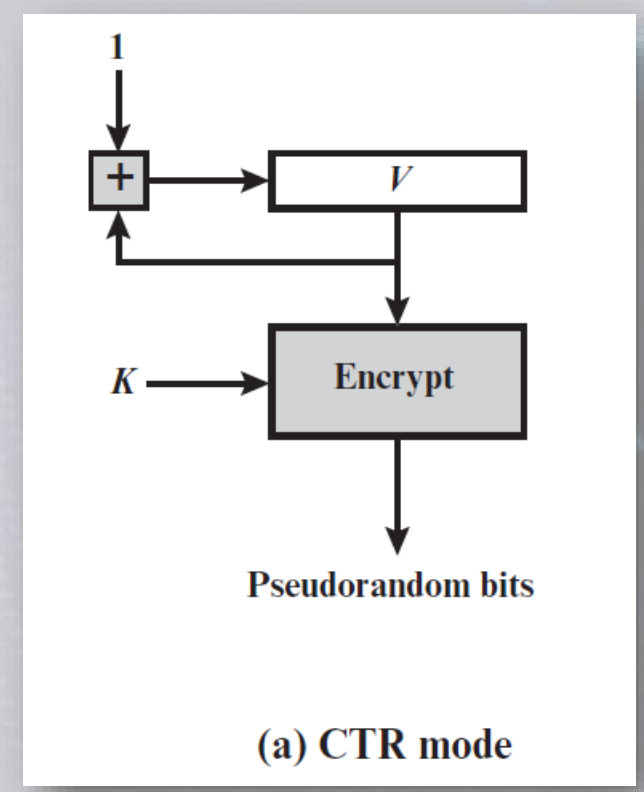
- Widely used modes: **CTR** mode and **OFB** mode.

- The seed consists of two parts: the ***encryption key value*** and ***a value V*** that will be **updated after each block** of pseudorandom numbers is generated.

- Thus, for AES-128, the seed consists of a 128-bit key and a 128-bit V value.

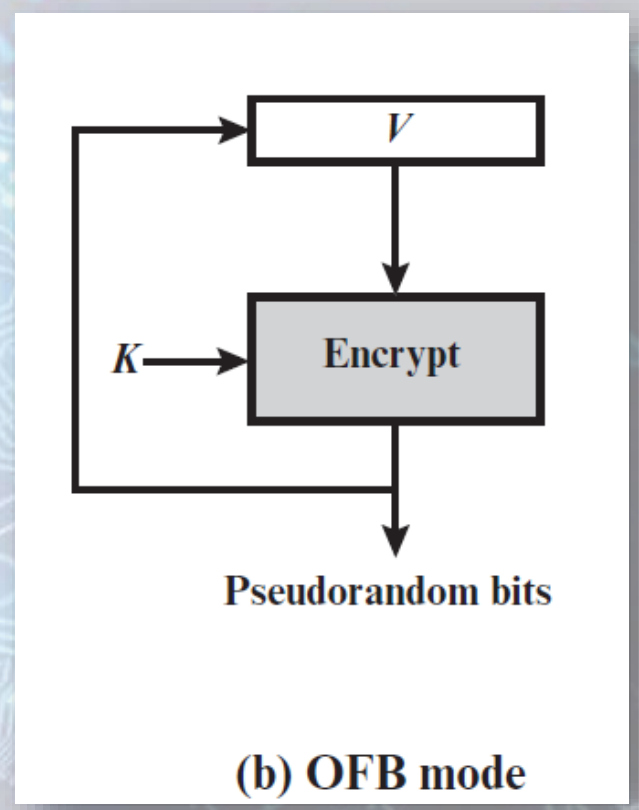
PRNG Using Block Cipher Modes of Operation

- The CTR algorithm for PRNG called CTR_DRBG:



(a) CTR mode

- The OFB algorithm for PRNG :



(b) OFB mode

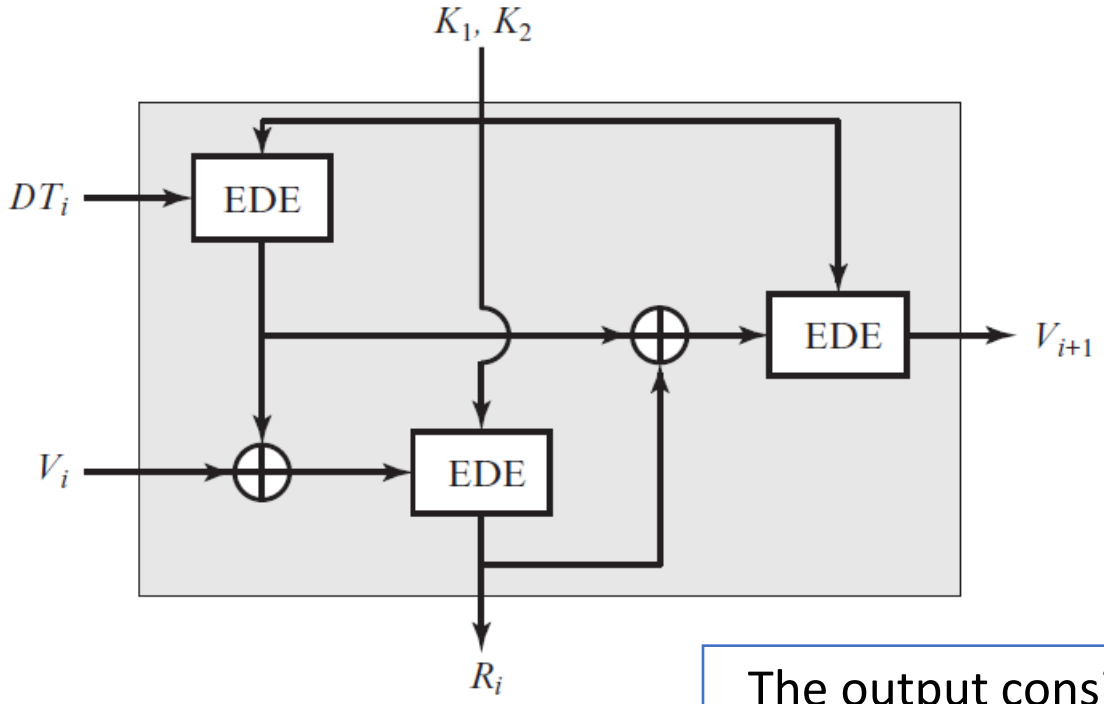
Why in CTR need to perform mod operation?

```
while (len (temp) < requested_number_of_bits) do
      V = (V + 1) mod 2^128
      output_block = E(Key, V)
      temp = temp || output_block
```

```
while (len (temp) < requested_number_of_bits) do
      V = E(Key, V)
      temp = temp || V
```

ANSI X9.17 PRNG

- One of the strongest (cryptographically speaking) PRNGs.
- The algorithm makes use of triple DES for encryption.



DT_i	Date/time value at the beginning of ith generation stage
V_i	Seed value at the beginning of ith generation stage
R_i	Pseudorandom number produced by the ith generation stage
K_1, K_2	DES keys used for each stage

V_i is initialized to some arbitrary value

where EDE($[K_1, K_2]$,X) refers to the sequence encrypt-decrypt-encrypt using two-key triple DES to encrypt X.

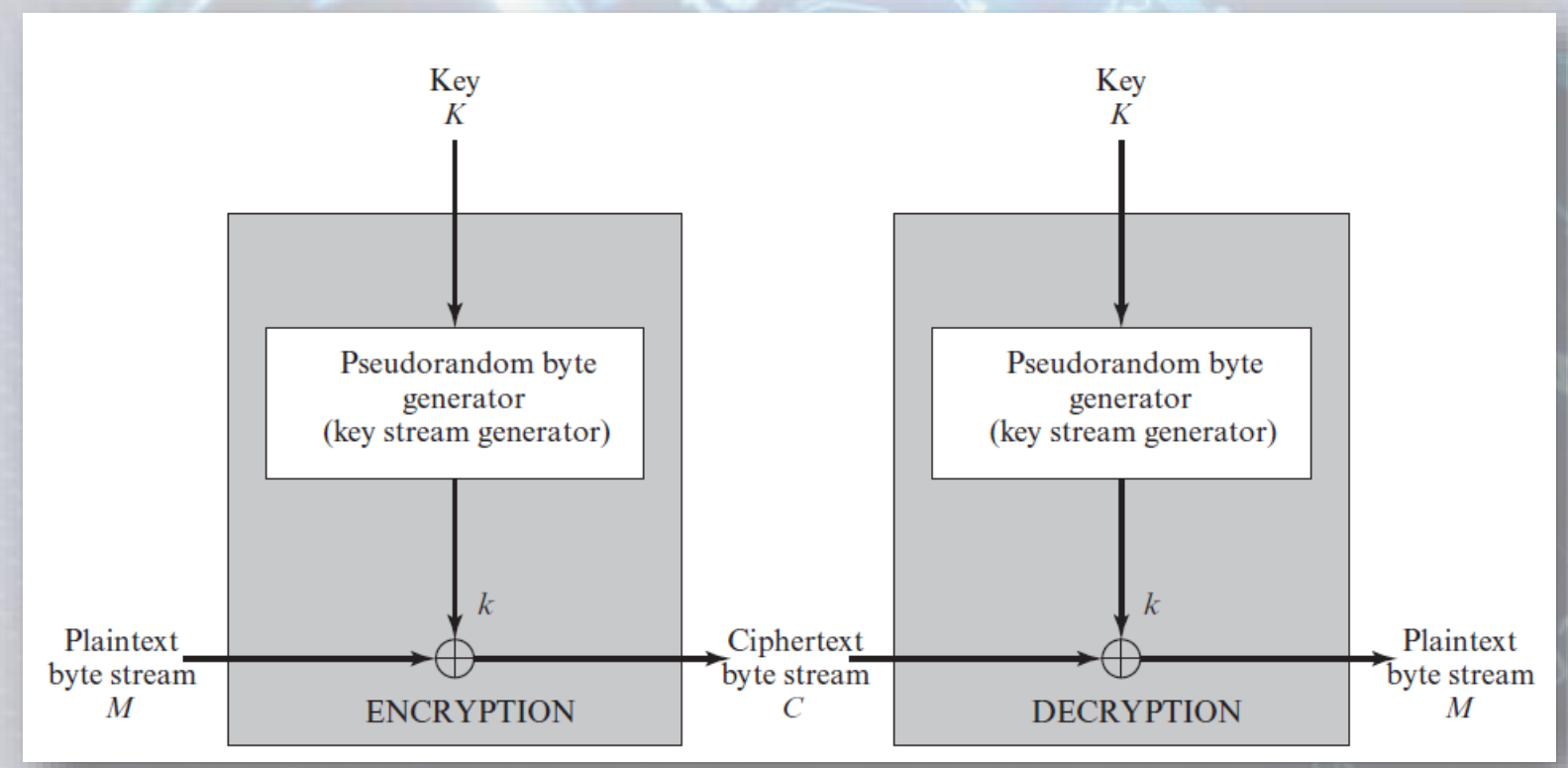
$$R_i = \text{EDE}([K_1, K_2], [V_i \oplus \text{EDE}([K_1, K_2], DT_i)])$$
$$V_{i+1} = \text{EDE}([K_1, K_2], [R_i \oplus \text{EDE}([K_1, K_2], DT_i)])$$

The output consists of a 64-bit pseudorandom number and a 64-bit seed value.

- Several factors contribute to the cryptographic strength of this method:
  - A total of nine DES encryptions.
  - Two independent inputs: DT and V
  - Even if a pseudorandom number Ri was compromised, it would be impossible to deduce the Vi+1 from the Ri, because an additional EDE operation is used to produce the Vi+1.

Stream Ciphers

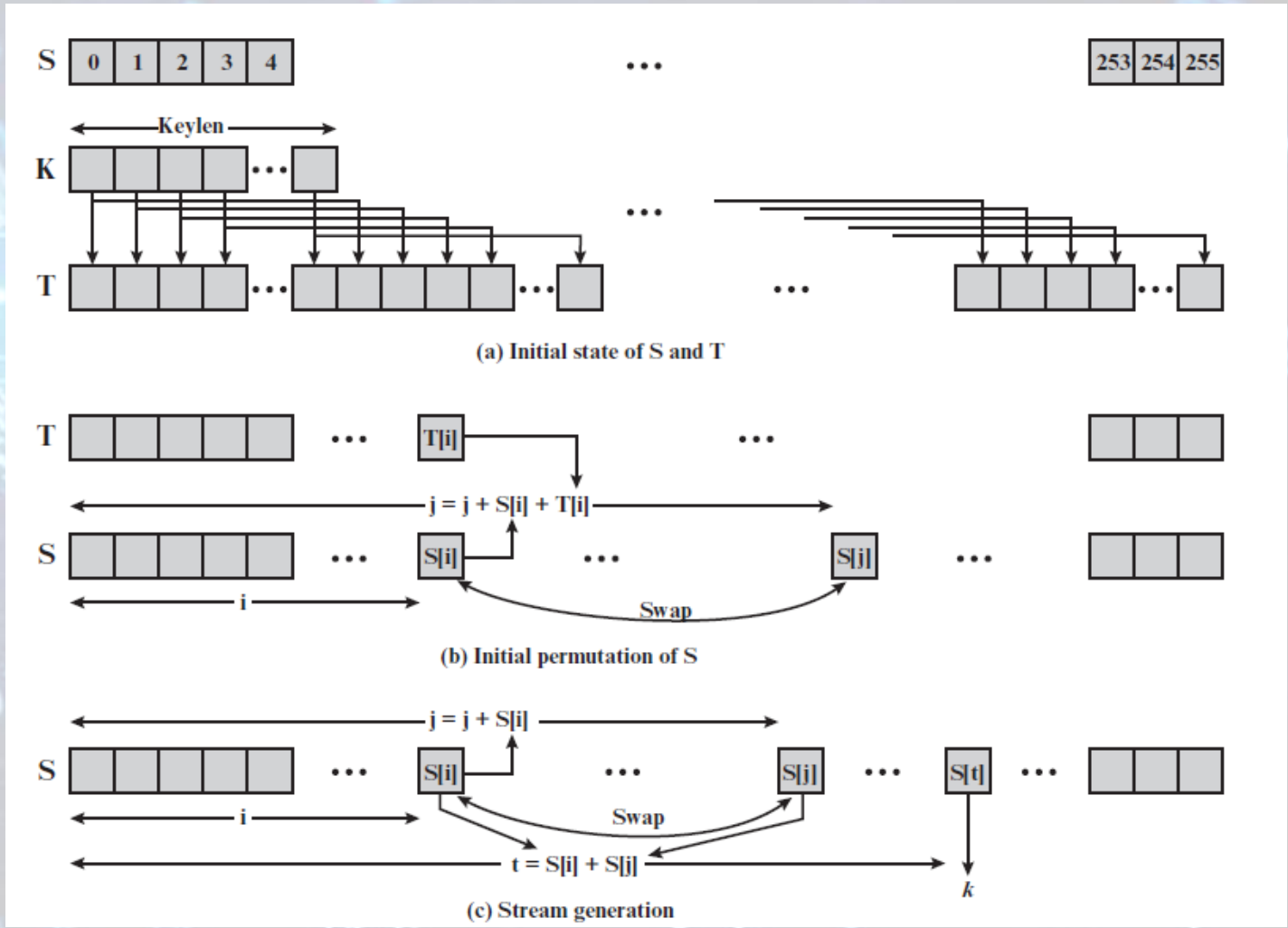
- A typical stream cipher encrypts plaintext **one byte** at a time, although a stream cipher may be designed to operate on **one bit** at a time or **on units larger than a byte** at a time.



RC4

- RC4 is a stream cipher with a variable key size and byte-oriented operations.

- A variable-length key of from 1 to 256 bytes (8 to 2048 bits) is used.
- At all times, S contains a permutation of all 8-bit numbers from 0 through 255.
- For encryption and decryption, a byte k is generated from S by selecting one of the 256 entries in a systematic fashion.



RC4

Initialization of S and T:

- The entries of S are set equal to the values from 0 through 255 in ascending order → S[0] = 0, S[1] = 1, … , S[255] = 255.
- A temporary vector, T, is also created. If the length of the key K is 256 bytes, then K is transferred to T. Otherwise, for a key of length keylen bytes, the first keylen elements of T are copied from K, and then K is repeated as many times as necessary to fill out T.

```
/* Initialization */
for i = 0 to 255 do
 S[i] = i;
 T[i] = K[i mod keylen];
```

Initial Permutation of S:

- Next, we use T to produce the initial permutation of S. This involves starting with S[0] and going through to S[255], and for each S[i], swapping S[i] with another byte in S according to a scheme dictated by T[i]:

```
/* Initial Permutation of S */
j = 0;
for i = 0 to 255 do
    j = (j + S[i] + T[i]) mod 256;
    Swap (S[i], S[j]);
```

RC4

Stream Generation:

- Once the S vector is initialized, the input key is no longer used.
- Stream generation involves cycling through all the elements of S[i], and for each S[i], swapping S[i] with another byte in S according to a scheme dictated by the current configuration of S.
- After S[255] is reached, the process continues, starting over again at S[0]:

```
/* Stream Generation */
i, j = 0;
while (true)
  i = (i + 1) mod 256;
  j = (j + S[i]) mod 256;
  Swap (S[i], S[j]);
  t = (S[i] + S[j]) mod 256;
  k = S[t];
```

- To encrypt, XOR the value k with the next byte of plaintext.
- To decrypt, XOR the value k with the next byte of ciphertext.


Thank You